SIMATS ENGINEERING

Department of Information Technology

ITA05 – Computer Vision

COMPUTER VISION – ASSIGNMENT 2 (CO2)

OLD PHOTOGRAPH RESTORATION SYSTEM

CO2: Apply image enhancement and segmentation techniques to restore damaged photographs and identify meaningful image regions.

Assignment Details	Information	Assignment Details	Information
Student Name	Yazhini R	Register Number	192521190
Department / Section	Information Technology	Academic Year / Semester	2025–2026
Faculty Name	Dr.Murali P	Nature of Assignment	Individual
Duration	1 Week	Total Marks	50
Date of Submission		Date of Evaluation	

SCENARIO

A college archive has a collection of old family and institutional photographs that have become faded and damaged over time. The photographs show loss of contrast, yellow/sepia colour shifts, scratches, dust, stains, ink marks, creases and uneven fading. Some images are slightly blurred, while others contain small torn or missing regions. The archive wants an Old Photograph Restoration System that (1) improves brightness, contrast and visual clarity while preserving original facial features and fine details, and (2) identifies damaged regions such as scratches, stains, dust spots, creases and torn areas so that they can be repaired or reviewed. The restored result must retain the historical appearance of the photograph and must not invent or alter important details.

You are the vision engineer. For every task, three candidate techniques are provided. Implement all three techniques on the same set of old photographs, compare their performance, select the most suitable technique, and justify the selection with experimental evidence. Pay special attention to preserving faces, clothing, background details and original photographic tone. State and justify any additional assumptions.

OPERATING DATA

Parameter	Operating Data
Image resolution	A4, 2480 × 3508 pixels (300 DPI)
Image format	8-bit grayscale
Text stroke width	2–4 pixels
Contrast	Faded ink is 30–50 grey levels darker than paper
Lighting	Uneven illumination with one-sided shadows
Noise	Gaussian noise ($\sigma \approx 6$) and 1% salt-and-pepper noise
Scanner noise	Horizontal stripes every 6 pixel rows
Processing time	Maximum 3 seconds per page
Target accuracy	Text segmentation Dice score ≥ 0.90
Hardware	4 CPU cores, 16 GB RAM, no GPU
Software	Python, OpenCV and NumPy

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS [4 Marks]

The old photographs contain faded areas and low contrast. Apply and compare three grey-level transformation techniques to recover visible details without losing highlights or shadow details.

Candidate Techniques

- A. Negative and Log Transformation
- B. Power-Law (Gamma) Transformation
- C. Piecewise-Linear Contrast Stretching

Questions

1. Implement all three transformations using Python/OpenCV on the same photographs.
2. Compare subject-to-background contrast, recovery of faint details, clipping effects and processing time.
3. Plot the transformation curves and corresponding input/output histograms.
4. Perform bit-plane slicing and identify which bit planes contain useful image details and which mainly contain noise or damage.
5. Analyse the effect of applying the same gamma value to both bright and dark photographs.
6. Compare a pretrained low-light or image-enhancement model with the selected transformation.

Evidence Expected: Transformation curves, before-and-after images, histograms, contrast comparison table and justified technique selection.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT [4 Marks]

Old photographs have different exposure levels, faded regions and uneven tonal distribution. Apply and compare three histogram-based enhancement techniques.

Candidate Techniques

- A. Global Histogram Equalization
- B. Histogram Matching
- C. Adaptive Histogram Equalization (CLAHE)

Questions

7. Implement all three techniques using Python/OpenCV.
8. Compare detail visibility, contrast improvement, noise amplification and correction of faded or overexposed regions.
9. Generate histograms and Cumulative Distribution Function (CDF) plots.
10. Analyse why global histogram equalization may fail when a photograph contains both very dark and very bright regions.
11. Experiment with different CLAHE tile sizes and clip limits.
12. Examine the effect of applying global histogram equalization to a photograph in which a large area has a nearly uniform background.

Evidence Expected: Histograms, CDF plots, before-and-after images, contrast/entropy comparison and justified parameter selection.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING [5 Marks]

Old photographs contain film grain, dust particles and salt-and-pepper-like damage. Apply three smoothing filters to reduce noise while preserving facial features, hair, clothing texture and fine edges.

Candidate Techniques

- A. Box (Averaging) Filter
- B. Gaussian Filter
- C. Median Filter

Questions

13. Implement all three filters using 3×3, 5×5 and 7×7 kernels.
14. Compare noise removal, detail preservation and processing time.
15. Calculate PSNR and SSIM for each filter.
16. Plot grayscale intensity profiles across selected facial edges, object boundaries or textured regions.
17. Demonstrate the difference between correlation and convolution using a 3×3 filter mask.
18. Analyse the consequences of using a large 7×7 filter on small details such as eyes, hair strands and narrow object boundaries.
19. Compare a classical filtering technique with a pretrained denoising model such as DnCNN.

Evidence Expected: Filtered images, PSNR/SSIM comparison table, intensity profiles and justified filter/kernel selection.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT [5 Marks]

Some old photographs are slightly blurred, making faces and object boundaries unclear. Apply three sharpening techniques to improve perceived detail without creating artificial edges.

Candidate Techniques

- A. Laplacian Sharpening
- B. Unsharp Masking / High-Boost Filtering
- C. Gradient-Based Sharpening

Questions

20. Implement all three sharpening techniques using Python/OpenCV.
21. Compare edge sharpness, noise amplification, halo effects and preservation of natural facial details.
22. Calculate the variance of Laplacian to measure image sharpness.
23. Plot intensity profiles across representative facial or object edges.
24. Explain why smoothing before sharpening may improve the output.
25. Analyse the effect of applying high-boost filtering with a large amplification factor to photographs containing scratches and grain.
26. Compare a traditional sharpening method with a pretrained deep-learning deblurring model.

Evidence Expected: Sharpened images, stroke profiles, sharpness/noise comparison and justified filter selection.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING [5 Marks]

Digitized old photographs may contain film grain, fine scratches and periodic scanning artefacts. Apply three frequency-domain low-pass filters to reduce unwanted high-frequency noise while preserving important image details.

Candidate Techniques

- A. Ideal Low-Pass Filter (ILPF)
- B. Butterworth Low-Pass Filter (BLPF)
- C. Gaussian Low-Pass Filter (GLPF)

Questions

27. Implement all three filters using Fourier Transform techniques.
28. Display the frequency spectrum and identify frequency patterns associated with grain, fine scratches or periodic scanning artefacts.
29. Compare the filters using different cut-off frequencies.
30. Measure ringing effects, loss of fine details, edge blurring and processing time.
31. Compare frequency-domain Gaussian filtering with spatial-domain Gaussian filtering in terms of detail preservation and noise reduction.
32. Analyse the effect of omitting padding before applying the Discrete Fourier Transform.
33. Evaluate edge/detail preservation before and after frequency-domain filtering using edge retention or a suitable image-quality measure.

Evidence Expected: Frequency spectra, filtered images, ringing/blur comparison and OCR performance table.

TASK 6 – FREQUENCY-DOMAIN HIGH-PASS FILTERING [4 Marks]

Old photographs may contain uneven fading, darkened corners and gradual illumination or colour changes. Apply three frequency-domain enhancement techniques to improve tonal consistency and reveal details.

Candidate Techniques

- A. Ideal / Butterworth High-Pass Filter
- B. Gaussian High-Pass Filtering with High-Frequency Emphasis
- C. Homomorphic Filtering

Questions

34. Implement all three techniques on shadow-affected photographs.
35. Compare detail sharpness, correction of uneven fading, noise amplification and preservation of overall photographic tone.
36. Measure local contrast and intensity variation in selected uniform regions.
37. Analyse the consequences of applying an ideal high-pass filter directly to an old photograph, especially around faces and strong edges.
38. Examine the effect of interchanging low-frequency and high-frequency gain parameters in homomorphic filtering.
39. Compare homomorphic filtering with a pretrained illumination-correction or low-light enhancement model.

Evidence Expected: High-pass filtered images, contrast/evenness comparison table and justified technique selection.

TASK 7 – POINT, LINE AND EDGE DETECTION [5 Marks]

The restoration system must locate scratches, dust spots, crease lines, stains and torn boundaries. Apply and compare three detection approaches.

Candidate Techniques

- A. Point and Line Detection Masks
- B. Gradient Edge Operators (Roberts, Prewitt and Sobel)
- C. Advanced Edge Detection (Marr–Hildreth and Canny)

Questions

40. Implement all three detection approaches using Python/OpenCV.
41. Detect dust spots, scratches, crease lines, stain boundaries and torn or missing regions.
42. Compare the techniques based on noise sensitivity, false detections and missed damage regions.
43. Calculate precision and recall using manually annotated ground-truth images.
44. Justify suitable thresholds for detecting small dust spots, thin scratches and major damaged boundaries.
45. Analyse why directional line masks may fail when scratches and creases occur at different orientations.
46. Compare a traditional edge detector with a deep-learning-based edge detector such as HED.

Evidence Expected: Detection maps, precision/recall comparison table and justified detector selection.

TASK 8 – IMAGE THRESHOLDING AND BINARIZATION [4 Marks]

The restoration system must separate damaged regions from relatively undamaged photograph areas. Fading, stains and uneven tones make this difficult. Apply and compare three thresholding techniques.

Candidate Techniques

- A. Basic Global Thresholding
- B. Otsu's Thresholding
- C. Adaptive Thresholding

Questions

47. Implement all three thresholding techniques using Python/OpenCV.
48. Compare their performance on lightly damaged, heavily faded and unevenly exposed photographs.
49. Generate binary damage masks and corresponding histograms.
50. Calculate Dice scores to evaluate damage-region segmentation accuracy.
51. Analyse why global thresholding may fail when damage and undamaged regions have overlapping intensity values.
52. Examine the effect of changing the block size and constant value in adaptive thresholding.
53. Compare traditional thresholding with a deep-learning-based image-damage segmentation model.

Evidence Expected: Binary images, annotated histograms, Dice score comparison and justified thresholding method.

TASK 9 – REGION-BASED SEGMENTATION AND INTEGRATED PIPELINE DESIGN [6 Marks]

The restoration system must identify connected damaged areas such as stains, scratches, creases and torn regions, while distinguishing the main subject from the background where needed. Apply three segmentation approaches and integrate the best techniques into a complete photograph-restoration system.

Candidate Techniques

- A. Region Growing
- B. Region Splitting and Merging
- C. Edge Linking and Boundary Detection

Questions

54. Implement all three segmentation techniques using Python/OpenCV.
55. Compare their ability to identify stains, ink marks, dust clusters, creases and torn or missing regions.
56. Calculate Intersection over Union (IoU) for damage-region detection and boundary localization error.
57. Develop an integrated photograph-restoration pipeline using the best techniques selected from Tasks 1–8.
58. Prepare a block diagram showing the complete processing sequence.
59. Measure the processing time of each stage and verify whether the complete system satisfies the 5-second-per-photo requirement.
60. Analyse the effect of selecting an incorrect seed point during region growing, particularly when a stain has a similar tone to the surrounding photograph.
61. Examine how overlapping scratches, creases and interrupted damage boundaries affect edge linking and region detection.
62. Compare traditional segmentation with a promptable model such as Segment Anything Model 2 (SAM 2) for identifying damaged regions.

Evidence Expected: Segmentation outputs, IoU comparison, corner localization results, integrated pipeline block diagram, stage-wise timing and final decision table.

TASK 10 – RESTORATION SYSTEM ADAPTATION FOR COLOUR AND HIGH-RESOLUTION PHOTOGRAPHS [8 Marks]

The archive now receives high-resolution colour photographs (4000 × 3000 pixels) of old prints, captured using mobile phones or flatbed scanners. Some images have strong sepia/yellow colour casts, faded faces, scratches and stains. A few photographs contain handwritten dates or notes on the reverse side that must not be erased. JPEG compression may introduce blocking artefacts. The restored image must be processed in less than 5 seconds using the same laboratory computer, and important original details must be preserved.

Candidate Techniques

- A. Retain – Use the existing restoration pipeline without modification.
- B. Modify – Resize images, retune enhancement and filtering parameters, and process luminance and colour channels separately to reduce colour casts while preserving natural skin tones.
- C. Redesign – Use colour restoration, frequency-domain damage reduction and inpainting for small damaged regions.

Questions

63. Implement all three strategies using Python/OpenCV on at least 15 old colour photographs.
64. Analyse the limitations of the existing pipeline when processing high-resolution, faded and damaged colour photographs.
65. Compare colour correction, detail enhancement, noise and damage removal, and preservation of facial and background details under the new conditions.
66. Measure damage-mask Dice score, colour fidelity, edge/detail preservation, perceptual quality and processing time. Where an undamaged reference is available, also report PSNR and SSIM.
67. Decide whether the existing pipeline should be RETAINED, MODIFIED or REDESIGNED.
68. Develop an improved pipeline that satisfies the restoration-quality and processing-time requirements.

69. Identify one operating condition under which the selected strategy may fail.
70. Explain how image restoration or inpainting models could support the improved system, and discuss the risk of generating details that were not present in the original photograph.

Evidence Expected: Comparison of three strategies, quantitative performance measurements, revised pipeline and justified RETAIN/MODIFY/REDESIGN decision.

MARK DISTRIBUTION

Task	Task Title	Marks
1	Grey Level Transformation Analysis	4
2	Histogram Processing and Enhancement	4
3	Spatial Filtering and Image Smoothing	5
4	Image Sharpening and Edge Enhancement	5
5	Frequency-Domain Low-Pass Filtering	5
6	Frequency-Domain High-Pass Filtering	4
7	Point, Line and Edge Detection	5
8	Image Thresholding and Binarization	4
9	Region-Based Segmentation and Integrated Pipeline Design	6
10	Restoration System Adaptation for Colour and High-Resolution Photographs	8
	TOTAL	50

COMMON SUBMISSION REQUIREMENTS

71. Python/OpenCV source code with comments.
72. Original and restored photograph images.
73. Results obtained by executing all three techniques for every task.
74. Comparison tables containing suitable performance metrics.
75. Graphs, histograms, intensity profiles and segmentation masks wherever applicable.
76. Advantages and limitations of each technique.
77. Final technique selection supported by experimental evidence.
78. An integrated photograph-restoration pipeline block diagram.
79. Processing-time measurements with units.
80. Final conclusion explaining the suitability of the selected techniques and how original details are preserved.

GENERAL INSTRUCTIONS

Use the same test photographs when comparing the three candidate techniques within a task. Clearly state the dataset size, preprocessing steps, parameter values and hardware/software environment. Do not report assumed or fabricated measurements as experimental results. If a pretrained model cannot be executed, clearly identify the comparison as a reasoned discussion rather than an experimental result. Do not invent facial features or historical details during restoration; clearly label any inpainting or generative reconstruction and retain the unedited original.

INTRODUCTION

Computer Vision is a field of artificial intelligence that enables computers to understand, process and analyse digital images. It uses image-processing techniques to improve image quality, identify objects, detect boundaries and extract meaningful information from photographs.

Old photographs are important historical records because they preserve memories of families, institutions, events and cultural activities. However, photographs may become damaged due to ageing, improper storage, exposure to sunlight, humidity, dust and physical handling.

Common problems in old photographs include fading, low contrast, yellow or sepia colour changes, scratches, dust spots, stains, creases, torn regions and image blurring. These problems reduce the visibility of important details, including faces, clothing, buildings and backgrounds.

The Old Photograph Restoration System is designed to improve the visual quality of damaged photographs and identify regions that require repair. The system applies image enhancement, filtering, sharpening, frequency-domain processing, edge detection, thresholding and segmentation techniques.

Python, OpenCV and NumPy are used to implement the image-processing operations. Different techniques are applied to the same photographs so that their performance can be compared fairly.

The main objective is to restore photographs while preserving their original appearance. The system must not generate false facial features or change important historical details. Any reconstructed region must be identified clearly.

SCENARIO AND PROBLEM DESCRIPTION

A college archive contains old family and institutional photographs that have become faded and damaged over time. The photographs contain several types of defects, including loss of contrast, yellow or sepia colour shifts, scratches, dust, stains, ink marks, creases and uneven fading.

Some photographs are slightly blurred, while others contain small torn or missing regions. These defects make it difficult to identify people and understand the original contents of the photographs.

The archive requires an Old Photograph Restoration System with two main functions.

The first function is image enhancement. It improves brightness, contrast and visual clarity while preserving facial features, clothing textures and background details.

The second function is damage identification. It detects scratches, stains, dust spots, creases and torn regions so that damaged areas can be repaired or reviewed.

The system compares three candidate techniques for each task. The techniques are applied to the same photographs, and their performance is evaluated using suitable image-quality measures.

The final technique is selected based on restoration quality, damage detection accuracy, processing time and preservation of original photographic details.

OPERATING DATA

Parameter	Operating data
Image resolution	A4, 2480 × 3508 pixels (300 DPI)
Image format	8-bit grayscale
Text stroke width	2–4 pixels
Contrast	Faded ink is 30–50 grey levels darker than paper
Lighting	Uneven illumination with one-sided shadows
Noise	Gaussian noise ($\sigma \approx 6$) and 1% salt-and-pepper noise
Scanner noise	Horizontal stripes every 6 pixel rows
Processing time	Maximum 3 seconds per page
Target accuracy	Text segmentation Dice score ≥ 0.90
Hardware	4 CPU cores, 16 GB RAM, no GPU
Software	Python, OpenCV and NumPy

These operating conditions define the requirements for implementing and evaluating the image-processing techniques.

TASK 1 – GREY LEVEL TRANSFORMATION ANALYSIS

Introduction

Grey-level transformation is an image-enhancement technique that changes the intensity values of pixels to improve the visibility of details. In an 8-bit grayscale image, pixel intensity values range from 0 to 255. A value of 0 represents black, while 255 represents white.

Old photographs often contain faded regions with low contrast. Grey-level transformations improve the visibility of these regions by modifying pixel intensities.

The three candidate techniques used in this task are:

1. Negative and Log Transformation.
2. Power-Law (Gamma) Transformation.
3. Piecewise-Linear Contrast Stretching.

The techniques are compared using the same input photographs.

1. Implement all three transformations using Python/OpenCV on the same photographs.

Answer:

The three grey-level transformations can be implemented using OpenCV and NumPy. The input photograph is first converted into grayscale format. Each transformation is then applied independently to the same input image.

A. Negative Transformation

Negative transformation reverses the intensity values of an image. Dark pixels become bright, and bright pixels become dark.

The mathematical expression is:

$s = L - 1 - r$

Where:

- s represents the output intensity.
- r represents the input intensity.
- L represents the number of possible intensity levels.
- For an 8-bit image, L = 256.

For example, an input intensity of 40 becomes 215.

Negative transformation is useful when important details appear as dark regions against a bright background. However, it changes the natural photographic appearance and is generally more useful for inspection than final restoration.

B. Log Transformation

Log transformation increases the visibility of low-intensity pixel values while compressing high-intensity values.

The mathematical expression is:

$s = c \log(1 + r)$

Where:

- s represents the output intensity.
- r represents the input intensity.
- c is a scaling constant.

For an 8-bit image, the scaling constant can be calculated as:

$c = 255 / \log(256)$

This transformation expands darker intensity values and compresses brighter intensity values. It can reveal faint details in dark areas of old photographs.

C. Power-Law (Gamma) Transformation

Gamma transformation modifies image brightness using a power-law relationship.

The mathematical expression is:

$s = 255 \times (r / 255)^{\gamma}$

Where:

- r represents the input intensity.
- s represents the output intensity.
- γ represents the gamma value.

When γ is less than 1, the image becomes brighter. When γ is greater than 1, the image becomes darker.

For faded photographs, a gamma value below 1 may improve the visibility of darker details. The gamma value must be selected carefully to avoid losing highlights.

D. Piecewise-Linear Contrast Stretching

Piecewise-linear contrast stretching improves image contrast by expanding a selected range of input intensity values.

The transformation uses control points to map the input intensity range to a new output range.

A general linear section is represented by:

$s = m(r - r_1) + s_1$

Where:

- r is the input intensity.
- s is the output intensity.
- r_1 and s_1 are the starting control-point coordinates.
- m is the slope of the selected line segment.

This technique is useful when the important photograph details are concentrated within a limited range of grey levels.

Python Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt

import time


# Read the original photograph

img = cv2.imread("old_photo.jpg", cv2.IMREAD_GRAYSCALE)
```

```
if img is None:
    raise FileNotFoundError("Image not found. Check the image path.")

# A. Negative transformation
start = time.perf_counter()

negative = 255 - img

negative_time = time.perf_counter() - start

# B. Log transformation
start = time.perf_counter()

c = 255 / np.log(256)
log_img = c * np.log1p(img.astype(np.float32))
log_img = np.clip(log_img, 0, 255).astype(np.uint8)

log_time = time.perf_counter() - start

# C. Gamma transformation
start = time.perf_counter()

gamma = 0.7
gamma_table = np.array([
    255 * ((i / 255) ** gamma)
    for i in range(256)
]).astype(np.uint8)

gamma_img = cv2.LUT(img, gamma_table)
```

```
gamma_time = time.perf_counter() - start

# D. Piecewise-linear contrast stretching
start = time.perf_counter()

r1, r2 = 60, 180

piecewise = np.zeros_like(img)

for i in range(256):
    if i < r1:
        piecewise[img == i] = 0
    elif i > r2:
        piecewise[img == i] = 255
    else:
        piecewise[img == i] = (
            (i - r1) * 255 / (r2 - r1)
        )

piecewise = piecewise.astype(np.uint8)

piecewise_time = time.perf_counter() - start

# Display results
images = [
    img,
    negative,
    log_img,
    gamma_img,
```

```
    piecewise
]

titles = [
    "Original",
    "Negative",
    "Log Transformation",
    "Gamma Transformation",
    "Piecewise Contrast Stretching"
]

plt.figure(figsize=(12, 8))

for i in range(5):
    plt.subplot(2, 3, i + 1)
    plt.imshow(images[i], cmap="gray")
    plt.title(titles[i])
    plt.axis("off")

plt.tight_layout()
plt.show()

print("Negative time:", negative_time)
print("Log time:", log_time)
print("Gamma time:", gamma_time)
print("Piecewise time:", piecewise_time)
```

Program Explanation:

First, the original photograph is loaded in grayscale format using cv2.imread(). The negative transformation reverses the pixel intensity values. Log transformation uses the natural logarithm to expand darker intensities. Gamma transformation applies a power-law equation using a gamma value of 0.7.

Piecewise-linear contrast stretching maps a selected intensity range to the complete output range. The processing time of each operation is measured using Python's time.perf_counter() function.

The five images are displayed together for visual comparison.

Expected Output:

The program displays the original photograph and four transformed images. The negative image shows reversed brightness values. The log-transformed image reveals darker regions, while gamma transformation increases brightness for the selected gamma value. Piecewise-linear stretching improves contrast within the selected intensity range.

The actual processing times depend on the image size and computer hardware.

2. Compare subject-to-background contrast, recovery of faint details, clipping effects and processing time.

Answer:

Subject-to-background contrast describes the difference in intensity between the important subject and its surrounding background. A photograph with sufficient contrast makes faces, clothing and other objects easier to identify.

Negative transformation reverses the original intensity relationship. It may make certain dark details easier to inspect, but it does not necessarily improve natural photographic contrast.

Log transformation expands dark intensity values. It can reveal faint details in shadowed regions, but it may reduce the apparent difference between bright regions.

Gamma transformation provides adjustable brightness correction. A suitable gamma value can improve dark facial regions without completely changing the image appearance.

Piecewise-linear contrast stretching expands a selected intensity range. It can improve the difference between the subject and background when their intensities fall within the selected range.

Clipping occurs when multiple input values are mapped to the minimum or maximum output intensity. Excessive clipping can remove details in shadows or highlights.

Technique	Subject contrast	Faint detail recovery	Clipping risk
Negative	Reverses contrast appearance	Useful for inspection	Low for simple inversion
Log	Improves dark-region visibility	Good in dark regions	Possible in highlights
Gamma	Adjustable brightness and contrast appearance	Good with suitable gamma	Depends on gamma
Piecewise-linear	Strong control over selected ranges	Good within selected range	High if control points are unsuitable

Processing time must be measured using the same image dimensions and computer configuration. No numerical processing-time result should be reported until the program is executed.

Conclusion:

Gamma transformation and piecewise-linear contrast stretching are suitable candidates for restoring faded photographs. The final selection should depend on the measured contrast improvement and preservation of facial and background details.

3. Plot the transformation curves and corresponding input/output histograms.

Answer:

A transformation curve represents the relationship between input pixel intensity and output pixel intensity. The horizontal axis represents input intensity, while the vertical axis represents output intensity.

A histogram displays the frequency of each grey-level intensity in an image. It helps identify whether pixel values are concentrated in dark, middle or bright regions.

The negative transformation produces a reversed intensity curve. The log transformation produces a curve that rises quickly for low input values and gradually increases for higher values. Gamma transformation changes the curve according to the gamma parameter. Piecewise-linear stretching produces connected straight-line sections.

Python Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    raise FileNotFoundError("Image not found.")


# Transformations

negative = 255 - img


c = 255 / np.log(256)

log_img = np.clip(

    c * np.log1p(img.astype(np.float32)),

    0, 255

).astype(np.uint8)


gamma = 0.7

gamma_img = np.clip(
```

```
    255 * ((img.astype(np.float32) / 255) ** gamma),
    0, 255
).astype(np.uint8)

piecewise = cv2.normalize(
    img, None, 0, 255, cv2.NORM_MINMAX
)

# Plot transformation curves
r = np.arange(256)

negative_curve = 255 - r
log_curve = c * np.log1p(r)
gamma_curve = 255 * ((r / 255) ** gamma)

plt.figure(figsize=(8, 5))
plt.plot(r, negative_curve, label="Negative")
plt.plot(r, log_curve, label="Log")
plt.plot(r, gamma_curve, label="Gamma")
plt.plot(r, r, label="Identity")
plt.xlabel("Input Intensity")
plt.ylabel("Output Intensity")
plt.title("Grey-Level Transformation Curves")
plt.legend()
plt.grid()
plt.show()

# Plot input and output histograms
images = [
    img,
```

```
    negative,
    log_img,
    gamma_img,
    piecewise
]

names = [
    "Original",
    "Negative",
    "Log",
    "Gamma",
    "Piecewise"
]

plt.figure(figsize=(12, 8))

for i, image in enumerate(images):
    plt.subplot(2, 3, i + 1)
    plt.hist(image.ravel(), bins=256, range=(0, 256))
    plt.title(names[i])
    plt.xlabel("Intensity")
    plt.ylabel("Frequency")

plt.tight_layout()
plt.show()
```

Explanation:

The first graph displays the transformation curves. The original identity curve represents an unchanged image. The other curves demonstrate how each transformation changes intensity values.

The histograms display the distribution of pixel intensities before and after enhancement. A histogram concentrated in a narrow intensity range indicates low contrast. A wider distribution may indicate improved contrast, but it does not automatically mean better restoration.

Conclusion:

Transformation curves explain the mathematical behaviour of enhancement techniques, while histograms help evaluate changes in intensity distribution.

4. Perform bit-plane slicing and identify which bit planes contain useful image details and which mainly contain noise or damage.

Answer:

Bit-plane slicing is a technique that separates an image into individual binary planes. Each pixel in an 8-bit grayscale image contains eight bits, numbered from bit 0 to bit 7.

The lower bit planes contain less significant intensity information, while the higher bit planes contain more significant intensity information.

For an 8-bit image, the bit-plane decomposition is represented as:

$I(x,y) = \Sigma [b_k(x,y) \times 2^k]$, for k = 0 to 7.

Where:

- I(x,y) represents the pixel intensity.
- b_k represents the binary value of bit plane k.
- k represents the bit position.

The lower bit planes, particularly bit 0 and bit 1, may contain fine intensity variations, sensor noise, film grain and small damage variations.

The higher bit planes, particularly bit 6 and bit 7, generally contain large-scale brightness and structural information. Facial regions, clothing and background shapes may be visible in these planes.

However, damage can also appear in higher bit planes when scratches or stains create strong intensity changes. Therefore, bit-plane importance must be examined using the actual photograph.

Python Program

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img = cv2.imread("old_photo.jpg", 0)
```

```
if img is None:
    raise FileNotFoundError("Image not found.")

plt.figure(figsize=(12, 8))

for bit in range(8):
    plane = (img >> bit) & 1
    plane = plane * 255

    plt.subplot(2, 4, bit + 1)
    plt.imshow(plane, cmap="gray")
    plt.title("Bit Plane " + str(bit))
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Explanation:

The image is shifted right by the selected bit position. The bitwise AND operation extracts the required binary plane. Multiplication by 255 converts binary values into visible black-and-white intensity values.

Conclusion:

Higher bit planes generally preserve the major image structure, while lower bit planes may contain fine variations and noise. Bit-plane slicing helps understand how image information is distributed across binary levels.

5. Analyse the effect of applying the same gamma value to both bright and dark photographs.

Answer:

Gamma transformation modifies brightness according to a power-law equation. Its effect depends on the original intensity distribution.

When gamma is less than 1, pixel intensities increase. This makes the photograph brighter. When gamma is greater than 1, pixel intensities decrease, making the photograph darker.

Applying the same gamma value to both bright and dark photographs may produce different results.

In a dark photograph, a gamma value below 1 may improve the visibility of faces and objects hidden in shadow regions. However, an excessive increase can make dark details appear washed out.

In a bright photograph, applying the same gamma value below 1 may increase brightness further. This can cause highlights to become excessively bright and reduce visible detail.

Similarly, applying a gamma value greater than 1 to a dark photograph may make important facial details difficult to identify. However, it may be useful for reducing excessive brightness in an overexposed photograph.

Therefore, gamma correction should be selected according to the exposure condition of the photograph.

Conclusion:

The same gamma value should not automatically be applied to every photograph. An adaptive approach based on brightness analysis provides better control over image enhancement.

6. Compare a pretrained low-light or image-enhancement model with the selected transformation.

Answer:

A pretrained image-enhancement model is a deep-learning model trained using a collection of images to improve visual quality. Such models learn relationships between low-quality input images and enhanced output images.

Traditional gamma transformation uses a mathematical equation to change pixel intensities. It is computationally simple and can be implemented using OpenCV without requiring a GPU.

A pretrained low-light enhancement model may improve brightness, local contrast and visibility in difficult lighting conditions. However, its performance depends on the training data and model architecture.

Comparison factor	Gamma transformation	Pretrained enhancement model
Working method	Mathematical intensity transformation	Learned neural-network processing
Hardware	CPU is sufficient	May require more processing resources
Processing complexity	Low	Usually higher
Parameter control	Direct gamma adjustment	Depends on the model
Facial preservation	Does not intentionally reconstruct faces	May alter or hallucinate details
Historical reliability	Predictable intensity mapping	Requires careful verification
Processing speed	Generally suitable for fast CPU processing	Model-dependent

A model such as Zero-DCE is an example of a low-light enhancement approach. It learns image-enhancement curves to improve low-light photographs. Its use in the archive should be evaluated carefully because enhancement models may change local appearance.

The original photograph must always be retained. The enhanced output should be compared with the original to verify that important facial and historical details remain unchanged.

Conclusion:

Gamma transformation is a suitable initial method because it is simple, fast and predictable. A pretrained model may provide additional enhancement, but it must be evaluated for processing time and historical detail preservation. The final selection should be based on actual photographs rather than assumed performance.

TASK 1 – OVERALL CONCLUSION

Grey-level transformations improve the visibility of faded photographs by modifying pixel intensities. Negative transformation is useful for inspection, log transformation improves dark-region visibility, gamma transformation provides adjustable brightness correction, and piecewise-linear contrast stretching improves selected intensity ranges.

Bit-plane slicing provides additional information about the distribution of image details and noise.

For the Old Photograph Restoration System, gamma transformation and piecewise-linear contrast stretching are suitable initial candidates. The final technique must be selected after comparing actual photographs, checking clipping, measuring contrast and verifying that faces and background details are preserved.

TASK 2 – HISTOGRAM PROCESSING AND ENHANCEMENT [4 Marks]

Introduction

Old photographs often suffer from fading, poor contrast, uneven brightness and different exposure levels. Histogram processing techniques improve the distribution of pixel intensities and make hidden details more visible.

The three techniques used in this task are:

1. Global Histogram Equalization
2. Histogram Matching
3. Contrast Limited Adaptive Histogram Equalization (CLAHE)

7. Implement all three techniques using Python/OpenCV.

A. Global Histogram Equalization

Global Histogram Equalization improves image contrast by redistributing pixel intensity values across the available grayscale range.

Program:

```
import cv2

import matplotlib.pyplot as plt


img = cv2.imread("old_photo.jpg", 0)


if img is None:

   print("Image not found")
```

```
    exit()


equalized = cv2.equalizeHist(img)


cv2.imshow("Original Image", img)

cv2.imshow("Global Histogram Equalization", equalized)


cv2.waitKey(0)

cv2.destroyAllWindows()
```

Explanation:

1. The photograph is loaded in grayscale.
2. cv2.equalizeHist() calculates and redistributes the intensity values.
3. Dark and faded regions may become more visible.
4. The enhanced photograph is displayed.

B. Histogram Matching

Histogram Matching adjusts the intensity distribution of an input photograph to resemble a reference photograph.

Program:

```
import cv2

import numpy as np


source = cv2.imread("old_photo.jpg", 0)

reference = cv2.imread("reference.jpg", 0)


if source is None or reference is None:

    print("Image not found")

    exit()


source_values, source_indices, source_counts = np.unique(

    source.ravel(), return_inverse=True, return_counts=True
```

```
)

reference_values, reference_counts = np.unique(
    reference.ravel(), return_counts=True
)

source_cdf = np.cumsum(source_counts).astype(float)
source_cdf /= source_cdf[-1]

reference_cdf = np.cumsum(reference_counts).astype(float)
reference_cdf /= reference_cdf[-1]

mapping = np.interp(
    source_cdf, reference_cdf, reference_values
)

matched = mapping[source_indices].reshape(source.shape)
matched = np.clip(matched, 0, 255).astype(np.uint8)

cv2.imshow("Original", source)
cv2.imshow("Histogram Matching", matched)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. The source and reference photographs are loaded.
2. Their histograms and cumulative distribution functions are calculated.
3. The source intensity distribution is mapped to the reference distribution.
4. The resulting image has a tonal distribution similar to the reference.

The reference image should have a suitable brightness and contrast level.

C. Adaptive Histogram Equalization (CLAHE)

CLAHE improves local contrast by processing smaller image regions and limiting excessive histogram amplification.

Program:

```
import cv2


img = cv2.imread("old_photo.jpg", 0)


if img is None:
    print("Image not found")
    exit()


clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)


result = clahe.apply(img)


cv2.imshow("Original Image", img)
cv2.imshow("CLAHE Enhanced Image", result)


cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. The image is divided into smaller tiles.
2. Histogram equalization is performed in each tile.
3. The clip limit controls excessive contrast enhancement.
4. Neighbouring tiles are combined smoothly.

8. Compare detail visibility, contrast improvement, noise amplification and correction of faded or overexposed regions.

The three methods produce different enhancement results.

Parameter	Global Histogram Equalization	Histogram Matching	CLAHE
Detail visibility	Improves overall visibility	Depends on reference image	Improves local details
Contrast	Improves global contrast	Matches reference contrast	Improves local contrast
Noise amplification	May increase noise	Depends on reference distribution	Can increase local noise
Faded regions	Improves general fading	Adjusts tones to reference	Reveals locally faded details
Overexposed regions	Cannot recover clipped details	Cannot recover missing details	Limited correction
Processing	Usually fast	Requires reference mapping	Slightly more processing

Analysis:

Global Histogram Equalization is useful when the photograph has generally poor contrast. However, it may increase unwanted noise.

Histogram Matching is useful when several photographs need a similar appearance. Its performance depends on selecting a suitable reference image.

CLAHE is suitable for photographs containing both dark and bright regions because it enhances local contrast. However, an excessively high clip limit can amplify grain and scratches.

9. Generate histograms and Cumulative Distribution Function (CDF) plots.

A histogram shows the frequency of each grayscale intensity value. A CDF represents the cumulative distribution of those intensity values.

Program:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    print("Image not found")

    exit()
```

```
equalized = cv2.equalizeHist(img)

clahe = cv2.createCLAHE(
    clipLimit=2.0,
    tileGridSize=(8, 8)
)

clahe_img = clahe.apply(img)

images = [
    ("Original", img),
    ("Global Equalization", equalized),
    ("CLAHE", clahe_img)
]

for name, image in images:
    histogram = cv2.calcHist(
        [image], [0], None, [256], [0, 256]
    ).flatten()

    cdf = np.cumsum(histogram)
    cdf = cdf / cdf[-1]

    plt.figure(figsize=(10, 4))

    plt.subplot(1, 2, 1)
    plt.plot(histogram)
    plt.title(name + " Histogram")
    plt.xlabel("Intensity")
    plt.ylabel("Frequency")
```

```
    plt.subplot(1, 2, 2)
    plt.plot(cdf)
    plt.title(name + " CDF")
    plt.xlabel("Intensity")
    plt.ylabel("Cumulative Probability")


    plt.tight_layout()
    plt.show()
```

Explanation:

- The histogram represents the distribution of image intensities.
- The CDF shows the cumulative probability of intensity values.
- A narrow histogram generally indicates a limited intensity range.
- Histogram equalization spreads the intensity distribution.
- CLAHE enhances contrast within local image regions.

Expected observation: The original faded photograph may have a narrow or uneven histogram. Global equalization generally spreads the intensity values, while CLAHE enhances local intensity differences.

10. Analyse why global histogram equalization may fail when a photograph contains both very dark and very bright regions.

Global Histogram Equalization calculates a single intensity transformation for the entire photograph.

When an image contains both very dark and very bright regions, a single global transformation may not enhance every region equally.

Reasons for failure:

1. **Uneven illumination:** Dark and bright regions have different intensity distributions.
2. **Loss of local details:** Small details in dark regions may remain difficult to identify.
3. **Noise amplification:** Noise in smooth areas may become more visible.
4. **Over-enhancement:** Bright areas may become excessively contrasted.
5. **Unequal contrast:** A single histogram transformation cannot independently adjust every region.

Example:

Consider an old family photograph with a dark background and a brightly illuminated face. Global equalization processes the entire photograph together. It may improve the background but produce excessive contrast around the face.

Conclusion: CLAHE is generally more suitable for photographs with uneven illumination because it enhances contrast locally.

11. Experiment with different CLAHE tile sizes and clip limits.

CLAHE has two important parameters:

- **Tile size:** Determines the size of the local image regions.
- **Clip limit:** Controls the maximum allowed histogram amplification.

Program:

```
import cv2
import matplotlib.pyplot as plt


img = cv2.imread("old_photo.jpg", 0)


if img is None:
    print("Image not found")
    exit()


settings = [
    (2.0, (4, 4)),
    (2.0, (8, 8)),
    (2.0, (16, 16)),
    (1.0, (8, 8)),
    (4.0, (8, 8))
]


plt.figure(figsize=(12, 9))


for i, (clip, tile) in enumerate(settings):
    clahe = cv2.createCLAHE(
        clipLimit=clip,
        tileGridSize=tile
```

```
    )

    result = clahe.apply(img)

    plt.subplot(2, 3, i + 1)
    plt.imshow(result, cmap="gray")
    plt.title(f"Clip={clip}, Tile={tile}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Parameter analysis:

Tile Size	Clip Limit	Expected Effect
4 × 4	2.0	Stronger local adaptation
8 × 8	2.0	Balanced local enhancement
16 × 16	2.0	Larger regions and smoother enhancement
8 × 8	1.0	Lower contrast amplification
8 × 8	4.0	Stronger enhancement and possible noise amplification

Conclusion: An 8 × 8 tile size with a clip limit of 2.0 is a reasonable starting configuration. The final selection must be based on visual inspection of the test photographs.

12. Examine the effect of applying global histogram equalization to a photograph in which a large area has a nearly uniform background.

A photograph may contain a large uniform background, such as a wall, sky or plain cloth.

When global histogram equalization is applied, the background pixels contribute significantly to the overall histogram.

Effects:

1. A large number of similar background pixels can influence the histogram distribution.
2. The contrast of the main subject may not improve sufficiently.
3. Small background intensity variations may become more visible.
4. Noise and unwanted marks may be enhanced.
5. The image may look unnatural if the transformation is too strong.

Conclusion: Global Histogram Equalization is less suitable when the photograph contains a large uniform background and important local details. CLAHE can provide better local enhancement, but its parameters must be selected carefully.

Task 2 – Final Comparison and Conclusion

Technique	Main Advantage	Main Limitation
Global Histogram Equalization	Simple and fast	May over-enhance noise
Histogram Matching	Produces consistent tonal appearance	Requires a suitable reference
CLAHE	Enhances local contrast	High clip limits may amplify noise

Selected Technique: CLAHE

CLAHE is selected as the preferred technique for unevenly faded old photographs because it enhances local contrast while limiting excessive amplification. Histogram Matching can also be used when a consistent tonal appearance is required across an archive.

TASK 3 – SPATIAL FILTERING AND IMAGE SMOOTHING [5 Marks]

Introduction

Old photographs may contain film grain, dust particles, salt-and-pepper noise and small unwanted marks. Spatial filtering reduces noise by processing pixels and their neighbouring pixels.

The three techniques used are:

1. Box (Averaging) Filter
2. Gaussian Filter
3. Median Filter

13. Implement all three filters using 3 × 3, 5 × 5 and 7 × 7 kernels.

A. Box Filter

The Box Filter replaces each pixel with the average intensity of the pixels in its neighbourhood.

Program:

```
import cv2


img = cv2.imread("old_photo.jpg", 0)


if img is None:
    print("Image not found")
    exit()
```

```
for k in [3, 5, 7]:
    result = cv2.blur(img, (k, k))

    cv2.imshow(f"Box Filter {k}x{k}", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

The filter calculates the average of neighbouring pixels. It reduces random noise but may blur important details such as eyes, hair and clothing edges.

B. Gaussian Filter

The Gaussian Filter uses a weighted average in which nearby pixels receive greater importance.

Program:

```
import cv2

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()

for k in [3, 5, 7]:
    result = cv2.GaussianBlur(
        img, (k, k), 0
    )

    cv2.imshow(f"Gaussian Filter {k}x{k}", result)

cv2.waitKey(0)
```

```
cv2.destroyAllWindows()
```

Explanation:

The Gaussian Filter gives greater weight to pixels near the centre. It reduces Gaussian noise and produces smoother images.

C. Median Filter

The Median Filter replaces each pixel with the median value of the pixels in its neighbourhood.

Program:

```
import cv2


img = cv2.imread("old_photo.jpg", 0)


if img is None:
    print("Image not found")
    exit()


for k in [3, 5, 7]:
    result = cv2.medianBlur(img, k)


    cv2.imshow(f"Median Filter {k}x{k}", result)


cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

The Median Filter sorts the neighbouring pixel values and selects the middle value. It is effective against salt-and-pepper noise and isolated dust-like spots.

14. Compare noise removal, detail preservation and processing time.

Filter	Noise Removal	Detail Preservation	Processing Time
Box Filter	Reduces random noise	Low for fine details	Generally low
Gaussian Filter	Effective for Gaussian noise	Better than simple averaging in many cases	Generally low

Median Filter	Very effective for salt-and-pepper noise	Preserves edges better	Depends on kernel size

Analysis:

- Box filtering smooths the image but may remove small details.
- Gaussian filtering reduces random variations while providing smooth results.
- Median filtering removes isolated extreme values without averaging them into surrounding pixels.

Larger kernels generally produce stronger smoothing but can remove important photographic details.

15. Calculate PSNR and SSIM for each filter.

PSNR and SSIM are image-quality measures used to compare a processed image with a reference image.

PSNR Formula:

PSNR is calculated using the Mean Squared Error (MSE).

[
MSE=\frac{1}{MN}\sum_{i=1}^{M}\sum_{j=1}^{N}
[I(i,j)-K(i,j)]^2
]

[
PSNR=10\log_{10}\left(\frac{MAX^2}{MSE}\right)
]

Where:

- M and N are image dimensions.
- I is the original reference image.
- K is the processed image.
- MAX is the maximum pixel intensity, normally 255.

SSIM Formula:

[
SSIM(x,y)=
\frac{(2\mu_x\mu_y+C_1)(2\sigma_{xy}+C_2)}
{(\mu_x^2+\mu_y^2+C_1)(\sigma_x^2+\sigma_y^2+C_2)}
]

SSIM compares luminance, contrast and structural similarity.

Program:

Install the required package:

pip install opencv-python scikit-image

import cv2

```
from skimage.metrics import peak_signal_noise_ratio
from skimage.metrics import structural_similarity

img = cv2.imread("clean_reference.jpg", 0)
noisy = cv2.imread("old_photo.jpg", 0)

if img is None or noisy is None:
    print("Image not found")
    exit()

if img.shape != noisy.shape:
    noisy = cv2.resize(
        noisy, (img.shape[1], img.shape[0])
    )

filters = {
    "Box": lambda x: cv2.blur(x, (3, 3)),
    "Gaussian": lambda x: cv2.GaussianBlur(x, (3, 3), 0),
    "Median": lambda x: cv2.medianBlur(x, 3)
}

for name, function in filters.items():
    result = function(noisy)

    psnr = peak_signal_noise_ratio(
        img, result, data_range=255
    )

    ssim = structural_similarity(
        img, result, data_range=255
```

```
)

print(name)
print("PSNR:", psnr)
print("SSIM:", ssim)
```

Note: A clean reference image is required for meaningful reference-based PSNR and SSIM measurements. If the original photograph is already noisy or damaged and no clean reference exists, these values should not be presented as restoration accuracy.

16. Plot grayscale intensity profiles across selected facial edges, object boundaries or textured regions.

An intensity profile displays how grayscale values change along a selected row or column of an image.

It helps analyse whether smoothing removes noise while preserving important edges.

Program:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()

box = cv2.blur(img, (3, 3))
gaussian = cv2.GaussianBlur(img, (3, 3), 0)
median = cv2.medianBlur(img, 3)

row = img.shape[0] // 2

plt.figure(figsize=(10, 5))

plt.plot(img[row, :], label="Original")
plt.plot(box[row, :], label="Box Filter")
```

```
plt.plot(gaussian[row, :], label="Gaussian Filter")

plt.plot(median[row, :], label="Median Filter")


plt.title("Grayscale Intensity Profile")

plt.xlabel("Pixel Position")

plt.ylabel("Intensity")

plt.legend()

plt.grid()

plt.show()
```

Explanation:

1. A grayscale photograph is loaded.
2. Three smoothing filters are applied.
3. The middle horizontal row is selected.
4. Pixel intensity values are plotted.
5. The curves are compared to understand smoothing and edge preservation.

Expected observation: The original profile may contain rapid intensity fluctuations caused by noise. Filtering reduces these fluctuations. Excessive smoothing can also reduce sharp changes at facial and object boundaries.

17. Demonstrate the difference between correlation and convolution using a 3 × 3 filter mask.

Correlation and convolution are operations used in image processing to apply a filter mask to an image.

Correlation:

In correlation, the filter mask is applied directly without rotating it.

Convolution:

In convolution, the filter mask is rotated by 180 degrees before applying it.

Consider the following 3 × 3 mask:

$$
H=
\begin{bmatrix}
1&2&3\\
4&5&6\\
7&8&9
\end{bmatrix}
$$

The rotated mask used in convolution is:

[
H_r=
\begin{bmatrix}
9&8&7\
6&5&4\
3&2&1
\end{bmatrix}
]

Program:

```
import cv2

import numpy as np


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    print("Image not found")

    exit()


mask = np.array([

    [1, 2, 3],

    [4, 5, 6],

    [7, 8, 9]

], dtype=np.float32)


correlation = cv2.filter2D(

    img, -1, mask

)


convolution_mask = cv2.flip(mask, -1)


convolution = cv2.filter2D(

    img, -1, convolution_mask

)
```

```
cv2.imshow("Original", img)

cv2.imshow("Correlation", correlation)

cv2.imshow("Convolution", convolution)


cv2.waitKey(0)

cv2.destroyAllWindows()
```

Difference:

Correlation	**Convolution**
Mask is applied directly	Mask is rotated by 180 degrees
Does not reverse the filter	Reverses the filter
Used in template matching and filtering	Common in linear system analysis
Equivalent to convolution for symmetric masks	Equivalent to correlation for symmetric masks

Conclusion: The output may differ for asymmetric masks. For symmetric masks, both operations produce the same result.

18. Analyse the consequences of using a large 7 × 7 filter on small details such as eyes, hair strands and narrow object boundaries.

A 7 × 7 filter processes a larger neighbourhood around every pixel. It provides stronger smoothing than 3 × 3 and 5 × 5 filters.

Advantages:

1. Removes more random noise.
2. Reduces visible film grain.
3. Produces smoother image regions.
4. Can reduce small unwanted variations.

Disadvantages:

1. Small eye details may become blurred.
2. Fine hair strands may disappear.
3. Narrow object boundaries may become less visible.
4. Facial expressions may lose clarity.
5. Small scratches may become less distinguishable from the surrounding area.

Conclusion: A 7 × 7 filter may be useful for heavily noisy smooth regions, but it is not always suitable for complete old photographs. A smaller kernel such as 3 × 3 is a safer starting point when facial and historical details must be preserved.

19. Compare a classical filtering technique with a pretrained denoising model such as DnCNN.

DnCNN is a deep-learning-based image denoising model. It learns noise patterns from training images and estimates the noise present in an input photograph.

Parameter	Classical Filtering	DnCNN
Technique	Mathematical filtering	Deep neural network
Training	Not required	Requires pretrained weights
Noise removal	Depends on filter and kernel	Learns noise patterns
Detail preservation	May blur fine details	Can preserve more structures
Hardware	Suitable for CPU	Inference may be slower on CPU
Complexity	Simple	More complex
Historical detail risk	Usually does not generate new content	May alter or suppress subtle details

Analysis:

A median filter is useful for salt-and-pepper noise, while Gaussian filtering is suitable for Gaussian noise. DnCNN can handle learned noise patterns, but its output must be inspected carefully because historical photographs contain unique details that may not be represented in its training data.

Conclusion: Classical filtering is suitable for a simple CPU-based restoration system. DnCNN can be considered when suitable pretrained weights and sufficient processing resources are available. A pretrained model must be executed before reporting experimental performance.

Task 3 – Final Comparison and Conclusion

Technique	Main Advantage	Main Limitation
Box Filter	Simple averaging operation	Blurs edges
Gaussian Filter	Effective for Gaussian noise	May remove fine details
Median Filter	Removes salt-and-pepper noise	May remove very thin details with large kernels

Selected Technique: Median Filter

The Median Filter is selected for photographs containing isolated dust particles and salt-and-pepper-like noise because it removes extreme pixel values while preserving edges better than ordinary averaging filters in many situations. Gaussian filtering can be used separately when the dominant noise is Gaussian.

TASK 4 – IMAGE SHARPENING AND EDGE ENHANCEMENT [5 Marks]

Introduction

Old photographs may become blurred because of ageing, poor scanning, camera movement or low-quality printing. Image sharpening improves the visibility of edges and fine details.

The three sharpening techniques are:

1. Laplacian Sharpening
2. Unsharp Masking / High-Boost Filtering
3. Gradient-Based Sharpening

20. Implement all three sharpening techniques using Python/OpenCV.

A. Laplacian Sharpening

The Laplacian operator detects rapid intensity changes in an image. It is a second-order derivative operator used to enhance fine details.

Program:

```
import cv2


img = cv2.imread("old_photo.jpg", 0)


if img is None:
    print("Image not found")
    exit()


laplacian = cv2.Laplacian(
    img, cv2.CV_64F
)


laplacian = cv2.convertScaleAbs(laplacian)


sharpened = cv2.addWeighted(
    img, 1.0, laplacian, -0.5, 0
)


cv2.imshow("Original", img)
```

```
cv2.imshow("Laplacian", laplacian)

cv2.imshow("Laplacian Sharpening", sharpened)


cv2.waitKey(0)

cv2.destroyAllWindows()
```

Explanation:

1. The grayscale image is loaded.
2. The Laplacian operator calculates second-order intensity changes.
3. The calculated edge information is combined with the original image.
4. Fine intensity changes become more visible.

B. Unsharp Masking / High-Boost Filtering

Unsharp masking sharpens an image by subtracting a blurred version from the original image.

Formula:

[
Mask=Original-Blurred
]

[
Sharpened=Original+k(Mask)
]

Where k is the sharpening strength.

Program:

```
import cv2


img = cv2.imread("old_photo.jpg", 0)


if img is None:
    print("Image not found")
    exit()


blurred = cv2.GaussianBlur(
    img, (5, 5), 1.0
)
```

```
mask = cv2.subtract(img, blurred)


sharpened = cv2.addWeighted(
    img, 1.5, mask, 1.0, 0
)


cv2.imshow("Original", img)
cv2.imshow("Unsharp Mask", mask)
cv2.imshow("Unsharp Masking", sharpened)


cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. A blurred image is created using Gaussian filtering.
2. The blurred image is subtracted from the original.
3. The difference represents high-frequency details.
4. The detail mask is added to the original image.
5. The final image appears sharper.

C. Gradient-Based Sharpening

Gradient operators calculate intensity changes in horizontal and vertical directions. Sobel operators are commonly used for this purpose.

Program:

```
import cv2
import numpy as np


img = cv2.imread("old_photo.jpg", 0)


if img is None:
    print("Image not found")
```

```
    exit()

gx = cv2.Sobel(
    img, cv2.CV_64F, 1, 0, ksize=3
)

gy = cv2.Sobel(
    img, cv2.CV_64F, 0, 1, ksize=3
)

magnitude = cv2.magnitude(gx, gy)

magnitude = cv2.normalize(
    magnitude, None, 0, 255, cv2.NORM_MINMAX
)

magnitude = magnitude.astype(np.uint8)

sharpened = cv2.addWeighted(
    img, 1.0, magnitude, 0.3, 0
)

cv2.imshow("Original", img)
cv2.imshow("Gradient Magnitude", magnitude)
cv2.imshow("Gradient Sharpening", sharpened)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. Sobel X detects horizontal intensity changes.

2. Sobel Y detects vertical intensity changes.
3. Gradient magnitude combines both directions.
4. The gradient information is added to the original image.
5. Edges become more visible.

21. Compare edge sharpness, noise amplification, halo effects and preservation of natural facial details.

Parameter	Laplacian	Unsharp Masking	Gradient-Based
Edge sharpness	High	Adjustable	Strong around edges
Noise amplification	Can be high	Depends on sharpening strength	Can amplify noise
Halo effects	Possible	Possible with high strength	Possible
Facial details	Requires careful tuning	Adjustable and controllable	Enhances boundaries
Main purpose	Enhance fine intensity changes	Improve perceived sharpness	Enhance edges

Analysis:

Laplacian sharpening detects fine intensity changes but may also amplify grain and scratches.

Unsharp masking provides control over sharpening strength and is useful for slightly blurred photographs.

Gradient-based sharpening enhances boundaries but may produce unwanted responses around noise and damaged areas.

Conclusion: Unsharp masking is a practical starting technique because its sharpening strength can be adjusted. However, excessive sharpening must be avoided to preserve natural facial appearance.

22. Calculate the variance of Laplacian to measure image sharpness.

Variance of Laplacian is a simple method used to estimate image sharpness.

A sharper image often contains stronger intensity changes, producing a higher Laplacian variance.

Formula:

[
Sharpness=Var(\nabla^2 I)
]

Where:

- I represents the grayscale image.
- (\nabla^2 I) represents the Laplacian.
- Var represents variance.

Program:

import cv2

```
img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()

laplacian = cv2.Laplacian(
    img, cv2.CV_64F
)

sharpness = laplacian.var()

print("Variance of Laplacian:", sharpness)
```

Explanation:

1. The image is converted to grayscale.
2. The Laplacian operator calculates second-order intensity changes.
3. The variance of the Laplacian is calculated.
4. A larger value often indicates stronger edge variation.

Limitation: A high variance does not always mean better image quality. Noise and scratches can also increase the variance. Therefore, the measurement should be combined with visual inspection.

23. Plot intensity profiles across representative facial or object edges.

Intensity profiles help compare edge transitions before and after sharpening.

Program:

```
import cv2
import matplotlib.pyplot as plt

img = cv2.imread("old_photo.jpg", 0)

if img is None:
```

```
    print("Image not found")
    exit()

lap = cv2.Laplacian(img, cv2.CV_64F)
lap = cv2.convertScaleAbs(lap)

blurred = cv2.GaussianBlur(img, (5, 5), 1.0)
mask = cv2.subtract(img, blurred)

unsharp = cv2.addWeighted(
    img, 1.5, mask, 1.0, 0
)

row = img.shape[0] // 2

plt.figure(figsize=(10, 5))

plt.plot(img[row, :], label="Original")
plt.plot(lap[row, :], label="Laplacian")
plt.plot(unsharp[row, :], label="Unsharp Masking")

plt.title("Intensity Profile Across Image")
plt.xlabel("Pixel Position")
plt.ylabel("Intensity")
plt.legend()
plt.grid()
plt.show()
```

Expected observation:

- The original image may have gradual transitions around blurred edges.
- Sharpening may produce steeper transitions.

- Excessive sharpening may create overshoots and unnatural intensity changes.

24. Explain why smoothing before sharpening may improve the output.

Smoothing before sharpening reduces unwanted noise and small intensity variations.

Sharpening operators enhance high-frequency components. Unfortunately, noise and scratches also contain high-frequency information.

If sharpening is applied directly to a noisy photograph, the noise may become more visible.

Advantages of smoothing before sharpening:

1. Reduces unwanted grain.
2. Prevents excessive noise amplification.
3. Produces cleaner edge enhancement.
4. Improves the visual quality of the output.
5. Helps preserve important facial and object boundaries.

Recommended sequence:

Original Image → Noise Reduction → Sharpening → Final Image

Conclusion: Applying suitable smoothing before sharpening improves the result because it reduces unwanted high-frequency noise before enhancing important edges.

25. Analyse the effect of applying high-boost filtering with a large amplification factor to photographs containing scratches and grain.

High-boost filtering increases the strength of details by adding an amplified high-frequency mask to the original image.

When a large amplification factor is used, both useful details and unwanted high-frequency components are enhanced.

Effects:

1. Fine scratches become more visible.
2. Film grain may become stronger.
3. Small dust marks may appear sharper.
4. Halos may form around strong edges.
5. Facial features may look unnatural.
6. Background texture may become excessively sharp.

Example:

If an old portrait contains a scratch across the cheek, high-boost filtering may make the scratch more prominent instead of removing it.

Conclusion: A large amplification factor is not suitable for damaged photographs. A moderate sharpening strength should be selected after noise reduction.

26. Compare a traditional sharpening method with a pretrained deep-learning deblurring model.

Traditional sharpening improves the appearance of existing edges. Deep-learning deblurring models learn image structures and blur patterns from training data.

Parameter	Traditional Sharpening	Deep-Learning Deblurring
Method	Mathematical filtering	Neural network
Training	Not required	Requires pretrained model
Edge enhancement	Direct enhancement	Learned restoration
Blur removal	Limited	Can handle complex blur
Noise handling	Depends on preprocessing	Depends on training
CPU performance	Usually suitable	May be computationally expensive
Historical accuracy	Does not intentionally reconstruct missing regions	May alter subtle details

Analysis:

Traditional methods such as unsharp masking are easy to implement and require limited resources. However, they cannot recover information that is completely lost due to severe blur.

Deep-learning deblurring models may produce better results for complex blur patterns, but their output must be checked carefully. They may modify fine facial or historical details.

Conclusion: Traditional sharpening is suitable for mild blur and CPU-based processing. Deep-learning deblurring can be considered for severe blur when suitable pretrained weights and hardware are available.

Task 4 – Final Comparison and Conclusion

Technique	Main Advantage	Main Limitation
Laplacian Sharpening	Enhances fine intensity changes	Can amplify noise
Unsharp Masking	Adjustable sharpening strength	Can produce halos
Gradient-Based Sharpening	Highlights object boundaries	May enhance scratches and grain

Selected Technique: Unsharp Masking

Unsharp masking is selected as the preferred traditional sharpening method because its strength can be adjusted. It improves the perceived clarity of slightly blurred photographs while providing control over excessive sharpening.

TASK 5 – FREQUENCY-DOMAIN LOW-PASS FILTERING [5 Marks]

Introduction

Frequency-domain filtering processes images using their frequency components.

Low-frequency components represent gradual intensity variations and broad image structures. High-frequency components represent fine details, edges, grain and certain types of scratches or scanning artefacts.

Low-pass filters reduce high-frequency components to remove unwanted noise and produce smoother images.

The three techniques used are:

1. Ideal Low-Pass Filter (ILPF)
2. Butterworth Low-Pass Filter (BLPF)
3. Gaussian Low-Pass Filter (GLPF)

27. Implement all three filters using Fourier Transform techniques.

Common Fourier Transform Process

1. Load the grayscale photograph.
2. Apply the Discrete Fourier Transform (DFT).
3. Shift the zero-frequency component to the centre.
4. Construct a low-pass filter mask.
5. Multiply the frequency spectrum by the mask.
6. Apply the inverse Fourier Transform.
7. Display the restored image.

A. Ideal Low-Pass Filter (ILPF)

The Ideal Low-Pass Filter allows frequencies within a specified cut-off distance and removes frequencies outside that distance.

Formula:

[
H(u,v)=
\begin{cases}
1,&D(u,v)\leq D_0\
0,&D(u,v)>D_0
\end{cases}
]

Where:

- D is the distance from the centre of the frequency spectrum.
- D_0 is the cut-off frequency.

Program:

```
import cv2
import numpy as np

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()

rows, cols = img.shape

dft = np.fft.fft2(img)
shift = np.fft.fftshift(dft)

crow = rows // 2
ccol = cols // 2

D0 = 40

mask = np.zeros((rows, cols), np.float32)

for u in range(rows):
    for v in range(cols):
        distance = np.sqrt(
            (u - crow) ** 2 + (v - ccol) ** 2
        )

        if distance <= D0:
            mask[u, v] = 1
```

```
filtered = shift * mask


inverse = np.fft.ifftshift(filtered)

result = np.fft.ifft2(inverse)


result = np.abs(result)

result = np.uint8(

    np.clip(result, 0, 255)

)


cv2.imshow("Original", img)

cv2.imshow("Ideal Low Pass", result)


cv2.waitKey(0)

cv2.destroyAllWindows()
```

B. Butterworth Low-Pass Filter (BLPF)

The Butterworth Low-Pass Filter provides a gradual transition between the passband and stopband.

Formula:

$$
H(u,v)=\frac{1}{1+(D(u,v)/D_0)^{2n}}
$$

Where:

- D is the frequency distance.
- D_0 is the cut-off frequency.
- n is the filter order.

Program:

```
import cv2

import numpy as np


img = cv2.imread("old_photo.jpg", 0)
```

```
if img is None:
    print("Image not found")
    exit()

rows, cols = img.shape

dft = np.fft.fft2(img)
shift = np.fft.fftshift(dft)

crow = rows // 2
ccol = cols // 2

D0 = 40
n = 2

u = np.arange(rows) - crow
v = np.arange(cols) - ccol

U, V = np.meshgrid(v, u)

D = np.sqrt(U**2 + V**2)

mask = 1 / (1 + (D / D0) ** (2 * n))

filtered = shift * mask

inverse = np.fft.ifftshift(filtered)
result = np.fft.ifft2(inverse)

result = np.abs(result)
```

```
result = np.uint8(
    np.clip(result, 0, 255)
)

cv2.imshow("Original", img)
cv2.imshow("Butterworth Low Pass", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

C. Gaussian Low-Pass Filter (GLPF)

The Gaussian Low-Pass Filter uses a Gaussian function to reduce high-frequency components smoothly.

Formula:

[
H(u,v)=e^{-D(u,v)^2/(2D_0^2)}
]

Program:

```
import cv2
import numpy as np

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()

rows, cols = img.shape

dft = np.fft.fft2(img)
shift = np.fft.fftshift(dft)

crow = rows // 2
```

```
ccol = cols // 2

D0 = 40

u = np.arange(rows) - crow
v = np.arange(cols) - ccol

U, V = np.meshgrid(v, u)

D = np.sqrt(U**2 + V**2)

mask = np.exp(
    -(D**2) / (2 * D0**2)
)

filtered = shift * mask

inverse = np.fft.ifftshift(filtered)
result = np.fft.ifft2(inverse)

result = np.abs(result)
result = np.uint8(
    np.clip(result, 0, 255)
)

cv2.imshow("Original", img)
cv2.imshow("Gaussian Low Pass", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

28. Display the frequency spectrum and identify frequency patterns associated with grain, fine scratches or periodic scanning artefacts.

A frequency spectrum represents the frequency components present in an image.

The Fourier Transform converts an image from the spatial domain into the frequency domain.

Program:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    print("Image not found")

    exit()


dft = np.fft.fft2(img)

shift = np.fft.fftshift(dft)


spectrum = 20 * np.log(

    np.abs(shift) + 1

)


plt.figure(figsize=(10, 5))


plt.subplot(1, 2, 1)

plt.imshow(img, cmap="gray")

plt.title("Original Image")

plt.axis("off")


plt.subplot(1, 2, 2)
```

```
plt.imshow(spectrum, cmap="gray")

plt.title("Frequency Spectrum")

plt.axis("off")


plt.tight_layout()

plt.show()
```

Frequency pattern analysis:

1. **Low frequencies:** Located near the centre of the shifted spectrum. They represent gradual brightness changes and broad structures.
2. **High frequencies:** Located farther from the centre. They represent fine details, edges and rapid intensity changes.
3. **Film grain:** Often produces distributed high-frequency energy.
4. **Fine scratches:** Can contribute directional frequency patterns.
5. **Periodic scanning artefacts:** May appear as distinct peaks or lines in the frequency spectrum.

Conclusion: The frequency spectrum helps identify unwanted frequency components and select an appropriate low-pass filter.

29. Compare the filters using different cut-off frequencies.

The cut-off frequency determines how much high-frequency information is retained.

A low cut-off frequency removes more high-frequency components, producing stronger smoothing. A high cut-off frequency retains more fine details.

Program:

```
import cv2

import numpy as np

import matplotlib.pyplot as plt


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    print("Image not found")

    exit()


rows, cols = img.shape
```

```
dft = np.fft.fft2(img)
shift = np.fft.fftshift(dft)

crow = rows // 2
ccol = cols // 2

cutoffs = [20, 40, 80]

plt.figure(figsize=(12, 8))

for i, D0 in enumerate(cutoffs):
    u = np.arange(rows) - crow
    v = np.arange(cols) - ccol

    U, V = np.meshgrid(v, u)
    D = np.sqrt(U**2 + V**2)

    mask = np.exp(
        -(D**2) / (2 * D0**2)
    )

    filtered = shift * mask

    inverse = np.fft.ifftshift(filtered)
    result = np.fft.ifft2(inverse)

    result = np.uint8(
        np.clip(np.abs(result), 0, 255)
    )
```

```
    plt.subplot(1, 3, i + 1)
    plt.imshow(result, cmap="gray")
    plt.title(f"Cut-off = {D0}")
    plt.axis("off")

plt.tight_layout()
plt.show()
```

Comparison:

Cut-off Frequency	Expected Result
20	Strong smoothing and greater detail loss
40	Moderate smoothing
80	More details retained but more noise may remain

Conclusion: The cut-off frequency must be selected carefully to balance noise reduction and detail preservation.

30. Measure ringing effects, loss of fine details, edge blurring and processing time.

The three filters have different frequency responses.

Parameter	Ideal LPF	Butterworth LPF	Gaussian LPF
Ringing	More noticeable	Depends on order	Minimal
Fine detail loss	Depends on cut-off	Depends on cut-off and order	Depends on cut-off
Edge blurring	Can be strong	Moderate to strong	Smooth transition
Frequency transition	Abrupt	Gradual	Smooth
Processing	Fourier transform required	Fourier transform required	Fourier transform required

Analysis:

The Ideal Low-Pass Filter has an abrupt frequency cut-off. This sharp transition can produce ringing effects around strong edges.

The Butterworth filter provides a smoother transition. Its order controls how quickly frequencies are reduced.

The Gaussian filter has a smooth frequency response and generally produces fewer ringing artefacts.

Processing-time measurement:

```
import cv2
```

```
import numpy as np
import time

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()

start = time.perf_counter()

dft = np.fft.fft2(img)
shift = np.fft.fftshift(dft)

rows, cols = img.shape
crow, ccol = rows // 2, cols // 2

u = np.arange(rows) - crow
v = np.arange(cols) - ccol
U, V = np.meshgrid(v, u)

D = np.sqrt(U**2 + V**2)
D0 = 40

mask = np.exp(-(D**2) / (2 * D0**2))

filtered = shift * mask
result = np.fft.ifft2(np.fft.ifftshift(filtered))
result = np.abs(result)
```

```
elapsed = time.perf_counter() - start


print("Processing Time:", elapsed, "seconds")
```

The measurement includes the Fourier transform, filtering and inverse transform for this implementation. Repeat the experiment for all three filters using the same photograph and computer.

31. Compare frequency-domain Gaussian filtering with spatial-domain Gaussian filtering in terms of detail preservation and noise reduction.

Gaussian filtering can be performed in both spatial and frequency domains.

Spatial-domain Gaussian filtering:

It uses a Gaussian kernel directly on image pixels.

Frequency-domain Gaussian filtering:

It applies a Gaussian low-pass mask to the Fourier Transform of the image.

Parameter	Spatial Gaussian	Frequency Gaussian
Processing domain	Spatial domain	Frequency domain
Main operation	Kernel convolution	Fourier transform and multiplication
Noise reduction	Reduces Gaussian noise	Reduces high-frequency noise
Detail preservation	Depends on kernel size and sigma	Depends on cut-off frequency
Implementation	Simple	More complex
Large images	Kernel operations may be efficient	Transform overhead may be significant

Analysis:

Both approaches can produce equivalent Gaussian smoothing when the parameters, boundary handling and filtering conditions are matched.

Spatial Gaussian filtering is easier to implement using OpenCV. Frequency-domain Gaussian filtering is useful when frequency analysis and frequency-selective processing are required.

Conclusion: Spatial Gaussian filtering is suitable for straightforward image smoothing, while frequency-domain Gaussian filtering provides more direct control over frequency components.

32. Analyse the effect of omitting padding before applying the Discrete Fourier Transform.

Padding is the process of adding extra pixels, usually zeros, around an image before applying frequency-domain filtering.

It helps reduce boundary-related artefacts during frequency-domain processing.

Effects of omitting padding:

1. Opposite image boundaries may interact during circular convolution.

2. Unwanted edge artefacts may appear.
3. Strong differences between image boundaries can create additional frequency components.
4. Filtering may produce visible discontinuities near image borders.
5. The output may contain ringing or boundary distortion.

Example:

If the left and right boundaries of a photograph have very different intensity values, circular processing may treat them as neighbouring regions. This can create unwanted edge effects.

Conclusion: Padding is useful when boundary artefacts are a concern. Appropriate cropping or border handling should be applied after inverse transformation to restore the original image dimensions.

33. Evaluate edge/detail preservation before and after frequency-domain filtering using edge retention or a suitable image-quality measure.

Edge preservation can be evaluated using edge detection and image-quality metrics.

One simple method is to compare the number of detected edge pixels before and after filtering.

Edge retention formula:

[
Edge\ Retention=
\frac{Number\ of\ retained\ edge\ pixels}
{Number\ of\ original\ edge\ pixels}
]

The comparison should use a consistent edge detector and threshold.

Program:

```
import cv2

import numpy as np


img = cv2.imread("old_photo.jpg", 0)


if img is None:

   print("Image not found")

   exit()


rows, cols = img.shape


dft = np.fft.fft2(img)
```

```
shift = np.fft.fftshift(dft)

crow, ccol = rows // 2, cols // 2

u = np.arange(rows) - crow
v = np.arange(cols) - ccol

U, V = np.meshgrid(v, u)
D = np.sqrt(U**2 + V**2)

D0 = 40

mask = np.exp(
    -(D**2) / (2 * D0**2)
)

filtered = shift * mask

result = np.fft.ifft2(
    np.fft.ifftshift(filtered)
)

result = np.uint8(
    np.clip(np.abs(result), 0, 255)
)

original_edges = cv2.Canny(img, 100, 200)
filtered_edges = cv2.Canny(result, 100, 200)

original_count = np.count_nonzero(original_edges)
```

```
filtered_count = np.count_nonzero(filtered_edges)


if original_count > 0:
    retention = filtered_count / original_count
else:
    retention = 0


print("Original edge pixels:", original_count)
print("Filtered edge pixels:", filtered_count)
print("Edge pixel count ratio:", retention)


cv2.imshow("Original", img)
cv2.imshow("Filtered", result)
cv2.imshow("Original Edges", original_edges)
cv2.imshow("Filtered Edges", filtered_edges)


cv2.waitKey(0)
cv2.destroyAllWindows()
```

Important limitation: The edge pixel count ratio is a simple measure, not a complete edge-retention accuracy score. A higher count may include noise or false edges. A stronger evaluation uses manually annotated edge ground truth and calculates precision, recall or F1-score.

Conclusion: Frequency-domain filtering should reduce unwanted high-frequency noise while retaining important facial and object boundaries. The cut-off frequency should be chosen using both visual inspection and edge-quality measurements.

Task 5 – Final Comparison and Conclusion

Technique	Main Advantage	Main Limitation
Ideal Low-Pass Filter	Simple frequency selection	Can produce ringing
Butterworth Low-Pass Filter	Adjustable transition	Filter order affects output
Gaussian Low-Pass Filter	Smooth frequency response	Can blur fine details

Selected Technique: Gaussian Low-Pass Filter

The Gaussian Low-Pass Filter is selected because it provides a smooth frequency transition and generally produces fewer ringing artefacts than the Ideal Low-Pass Filter. It is suitable for reducing high-frequency grain while maintaining a natural appearance when an appropriate cut-off frequency is selected.

OVERALL CONCLUSION – TASKS 2 TO 5

The four tasks demonstrate different image enhancement and restoration methods for old photographs.

1. **Task 2 – Histogram Processing:** CLAHE improves local contrast in unevenly faded photographs.
2. **Task 3 – Spatial Filtering:** Median filtering is effective for salt-and-pepper noise and isolated dust-like marks.
3. **Task 4 – Image Sharpening:** Unsharp masking improves the perceived clarity of slightly blurred photographs.
4. **Task 5 – Frequency-Domain Low-Pass Filtering:** Gaussian Low-Pass Filtering reduces high-frequency noise with fewer ringing artefacts.

These techniques can be combined in a restoration pipeline:

Original Photograph → Histogram Enhancement → Noise Reduction → Sharpening → Frequency-Domain Filtering → Restored Photograph

The order and parameters should be adjusted according to the type of damage. Excessive enhancement, smoothing or sharpening may remove important facial, clothing and background details. Therefore, the original photograph must always be retained for comparison.

EXPERIMENTAL ENVIRONMENT

Parameter	**Configuration**
Programming Language	Python
Libraries	OpenCV, NumPy, Matplotlib
Image Format	8-bit grayscale
Image Resolution	2480 × 3508 pixels
Hardware	4 CPU cores, 16 GB RAM
GPU	Not available
Input	Old damaged photographs

Output	Enhanced and restored photographs

34. Implement all three techniques on shadow-affected photographs.

A. Ideal / Butterworth High-Pass Filter

A High-Pass Filter allows high-frequency components to pass and reduces low-frequency components.

Ideal High-Pass Filter formula:

$$
H(u,v)=
\begin{cases}
0,&D(u,v)\leq D_0\\
1,&D(u,v)>D_0
\end{cases}
$$

Program:

```
import cv2

import numpy as np


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    print("Image not found")

    exit()


rows, cols = img.shape


dft = np.fft.fft2(img)

shift = np.fft.fftshift(dft)


crow, ccol = rows // 2, cols // 2


u = np.arange(rows) - crow

v = np.arange(cols) - ccol
```

```
U, V = np.meshgrid(v, u)


D = np.sqrt(U**2 + V**2)


D0 = 30


ideal_mask = np.ones((rows, cols), np.float32)
ideal_mask[D <= D0] = 0


filtered = shift * ideal_mask


result = np.fft.ifft2(
    np.fft.ifftshift(filtered)
)


result = np.abs(result)
result = np.uint8(np.clip(result, 0, 255))


cv2.imshow("Original", img)
cv2.imshow("Ideal High Pass", result)


cv2.waitKey(0)
cv2.destroyAllWindows()
```

Butterworth High-Pass Filter formula:

$$
H(u,v)=\frac{1}{1+(D_0/D(u,v))^{2n}}
$$

Program:

```
import cv2
import numpy as np
```

```
img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()

rows, cols = img.shape

dft = np.fft.fft2(img)
shift = np.fft.fftshift(dft)

crow, ccol = rows // 2, cols // 2

u = np.arange(rows) - crow
v = np.arange(cols) - ccol
U, V = np.meshgrid(v, u)

D = np.sqrt(U**2 + V**2)

D0 = 30
n = 2

mask = np.zeros_like(D)
nonzero = D > 0

mask[nonzero] = 1 / (
    1 + (D0 / D[nonzero]) ** (2 * n)
)

filtered = shift * mask
```

```
result = np.fft.ifft2(
    np.fft.ifftshift(filtered)
)

result = np.abs(result)
result = np.uint8(np.clip(result, 0, 255))

cv2.imshow("Original", img)
cv2.imshow("Butterworth High Pass", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. The photograph is loaded in grayscale.
2. Fourier Transform converts it into the frequency domain.
3. A high-pass filter mask is created.
4. Low-frequency components are reduced.
5. Inverse Fourier Transform produces the output.

B. Gaussian High-Pass Filtering with High-Frequency Emphasis

Gaussian High-Pass Filtering reduces low-frequency components smoothly. High-Frequency Emphasis adds a controlled amount of the original low-frequency information to the high-pass result.

Formula:

$$H_{HP}(u,v)=1-e^{-D^2/(2D_0^2)}$$

$$H_{HFE}(u,v)=a+bH_{HP}(u,v)$$

Where:

- a represents low-frequency gain.
- b represents high-frequency gain.

- D_0 represents cut-off frequency.

Program:

```
import cv2
import numpy as np

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()

rows, cols = img.shape

dft = np.fft.fft2(img)
shift = np.fft.fftshift(dft)

crow, ccol = rows // 2, cols // 2

u = np.arange(rows) - crow
v = np.arange(cols) - ccol
U, V = np.meshgrid(v, u)

D = np.sqrt(U**2 + V**2)

D0 = 30

high_pass = 1 - np.exp(
    -(D**2) / (2 * D0**2)
)
```

```
a = 0.5
b = 1.5

emphasis = a + b * high_pass

filtered = shift * emphasis

result = np.fft.ifft2(
    np.fft.ifftshift(filtered)
)

result = np.abs(result)
result = np.uint8(np.clip(result, 0, 255))

cv2.imshow("Original", img)
cv2.imshow("High Frequency Emphasis", result)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

1. Fourier Transform is applied.
2. A Gaussian high-pass mask is generated.
3. Low-frequency and high-frequency gains are selected.
4. The emphasis mask is multiplied with the spectrum.
5. The inverse transform produces the enhanced image.

C. Homomorphic Filtering

Homomorphic Filtering is used to improve uneven illumination and image contrast.

It separates the illumination and reflectance components of an image.

Image model:

[
f(x,y)=i(x,y)\times r(x,y)
]

Where:

- i(x,y) represents illumination.
- r(x,y) represents reflectance.

Taking logarithms converts multiplication into addition.

Program:

```
import cv2

import numpy as np


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    print("Image not found")

    exit()


image = img.astype(np.float32) + 1


log_image = np.log(image)


dft = np.fft.fft2(log_image)

shift = np.fft.fftshift(dft)


rows, cols = img.shape


crow, ccol = rows // 2, cols // 2


u = np.arange(rows) - crow

v = np.arange(cols) - ccol

U, V = np.meshgrid(v, u)
```

```
D = np.sqrt(U**2 + V**2)

D0 = 30
gamma_l = 0.5
gamma_h = 1.5
c = 1.0

H = (
    gamma_h - gamma_l
) * (
    1 - np.exp(-c * (D**2) / (D0**2))
) + gamma_l

filtered = shift * H

inverse = np.fft.ifft2(
    np.fft.ifftshift(filtered)
)

result = np.exp(np.real(inverse)) - 1

result = cv2.normalize(
    result, None, 0, 255, cv2.NORM_MINMAX
)

result = result.astype(np.uint8)

cv2.imshow("Original", img)
cv2.imshow("Homomorphic Filtering", result)
```

```
cv2.waitKey(0)

cv2.destroyAllWindows()
```

Explanation:

1. The input image is converted to floating-point values.
2. Logarithmic transformation separates illumination and reflectance effects.
3. Fourier Transform is applied.
4. A homomorphic filter is generated.
5. The filtered image is converted back using the inverse transform and exponential operation.

35. Compare detail sharpness, correction of uneven fading, noise amplification and preservation of overall photographic tone.

Parameter	Ideal / Butterworth HPF	Gaussian HFE	Homomorphic Filtering
Detail sharpness	Enhances edges	Enhances fine details	Improves local details
Uneven fading correction	Limited	Limited	More suitable
Noise amplification	Can be high	Depends on gain	Possible
Tone preservation	May remove broad tonal information	Adjustable	Adjustable
Main application	Edge enhancement	Detail enhancement	Illumination correction

Analysis:

Ideal and Butterworth filters enhance high-frequency components but may remove important low-frequency information.

Gaussian High-Frequency Emphasis provides better control over the enhancement of details.

Homomorphic Filtering is more suitable for uneven illumination because it adjusts the illumination component while retaining reflectance information.

36. Measure local contrast and intensity variation in selected uniform regions.

Local contrast can be measured using standard deviation within a selected image region.

Formula:

$$
[
Local\ Contrast=\sigma
]
$$

A higher standard deviation indicates greater intensity variation.

Program:

```
import cv2

import numpy as np


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    print("Image not found")

    exit()


# Select a region of interest

roi = img[50:150, 50:150]


mean_value = np.mean(roi)

std_value = np.std(roi)

min_value = np.min(roi)

max_value = np.max(roi)


print("Mean Intensity:", mean_value)

print("Local Contrast:", std_value)

print("Minimum Intensity:", min_value)

print("Maximum Intensity:", max_value)
```

Explanation:

1. A uniform region is selected.
2. Mean intensity is calculated.
3. Standard deviation measures intensity variation.
4. Minimum and maximum values indicate the intensity range.

The same region should be selected in the original and processed photographs for a fair comparison.

37. Analyse the consequences of applying an ideal high-pass filter directly to an old photograph, especially around faces and strong edges.

An Ideal High-Pass Filter removes low-frequency components and retains high-frequency information.

Effects:

1. Smooth facial regions may become dark or disappear.
2. Fine edges become more prominent.
3. Noise and scratches may become stronger.
4. Strong boundaries may show ringing.
5. Overall brightness and photographic tone may be lost.
6. The resulting image may look unnatural.

Conclusion:

Applying an Ideal High-Pass Filter directly to the complete photograph is not recommended for final restoration. It is more useful for analysing edges and high-frequency details. High-frequency emphasis or homomorphic filtering provides more control.

38. Examine the effect of interchanging low-frequency and high-frequency gain parameters in homomorphic filtering.

Homomorphic filtering uses two gain parameters:

- Gamma L controls low-frequency information.
- Gamma H controls high-frequency information.

Normally, gamma L is selected below 1 and gamma H is selected above 1.

Case 1: Normal configuration

[
\gamma_L=0.5,\quad \gamma_H=1.5
]

This reduces gradual illumination variations and enhances reflectance details.

Case 2: Interchanged configuration

[
\gamma_L=1.5,\quad \gamma_H=0.5
]

This increases low-frequency information and reduces high-frequency information.

Configuration	Expected Effect
Low gain = 0.5, High gain = 1.5	Reduces illumination variation and enhances details
Low gain = 1.5, High gain = 0.5	Emphasizes gradual intensity changes and suppresses fine details

Conclusion: Interchanging the parameters changes the behaviour of the filter. The normal configuration is generally more appropriate for uneven illumination correction and detail enhancement.

39. Compare homomorphic filtering with a pretrained illumination-correction or low-light enhancement model.

Parameter	Homomorphic Filtering	Deep-Learning Enhancement
Method	Frequency-domain mathematical technique	Neural network
Training	Not required	Requires pretrained weights
Illumination correction	Controls low-frequency variation	Learns complex illumination patterns
Detail enhancement	Controlled by filter parameters	Depends on training
Hardware	Suitable for CPU implementation	May require greater processing power
Historical detail preservation	Does not intentionally generate content	May alter or reconstruct subtle details

Analysis:

Homomorphic Filtering is simple to implement and does not require training data. It is useful for correcting gradual illumination changes.

A pretrained low-light enhancement model may handle more complex lighting conditions. However, the output must be examined for changes to facial features, clothing and background details.

Conclusion: Homomorphic Filtering is selected for the basic CPU-based restoration system because it provides illumination correction without requiring a pretrained model. Deep-learning methods may be evaluated as an additional option.

Task 6 – Final Comparison and Conclusion

Technique	Main Advantage	Main Limitation
Ideal / Butterworth HPF	Enhances high-frequency details	May remove overall image tone
Gaussian HFE	Adjustable detail enhancement	Can amplify scratches and noise
Homomorphic Filtering	Corrects uneven illumination	Requires careful gain selection

Selected Technique: Homomorphic Filtering

Homomorphic Filtering is selected because it is designed to improve uneven illumination and tonal consistency. It is useful for photographs with darkened corners and gradual fading. Its gain parameters should be adjusted carefully to preserve the natural appearance of the photograph.

TASK 7 – POINT, LINE AND EDGE DETECTION [5 Marks]

Introduction

Old photographs may contain dust spots, scratches, crease lines, stains and torn boundaries. Detection techniques identify these damaged regions so that they can be reviewed or repaired.

The three candidate approaches are:

1. Point and Line Detection Masks
2. Gradient Edge Operators
3. Advanced Edge Detection

40. Implement all three detection approaches using Python/OpenCV.

A. Point and Line Detection Masks

Point detection identifies isolated intensity changes. Line detection identifies structures with a particular orientation.

Program:

```
import cv2

import numpy as np


img = cv2.imread("old_photo.jpg", 0)


if img is None:

    print("Image not found")

    exit()


point_kernel = np.array([

    [-1, -1, -1],

    [-1,  8, -1],

    [-1, -1, -1]

], dtype=np.float32)


line_horizontal = np.array([

    [-1, -1, -1],

    [ 2,  2,  2],

    [-1, -1, -1]

], dtype=np.float32)


line_vertical = np.array([
```

```
    [-1,  2, -1],
    [-1,  2, -1],
    [-1,  2, -1]
], dtype=np.float32)

point_result = cv2.filter2D(
    img, cv2.CV_32F, point_kernel
)

horizontal = cv2.filter2D(
    img, cv2.CV_32F, line_horizontal
)

vertical = cv2.filter2D(
    img, cv2.CV_32F, line_vertical
)

point_result = cv2.convertScaleAbs(point_result)
horizontal = cv2.convertScaleAbs(horizontal)
vertical = cv2.convertScaleAbs(vertical)

cv2.imshow("Point Detection", point_result)
cv2.imshow("Horizontal Lines", horizontal)
cv2.imshow("Vertical Lines", vertical)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

Point masks identify isolated changes in intensity. Directional masks detect lines aligned with a selected orientation.

B. Gradient Edge Operators – Roberts, Prewitt and Sobel

Gradient operators detect boundaries by calculating intensity changes.

Program:

```
import cv2
import numpy as np


img = cv2.imread("old_photo.jpg", 0)


if img is None:
    print("Image not found")
    exit()


# Roberts operators
roberts_x = np.array([
    [1, 0],
    [0, -1]
], dtype=np.float32)


roberts_y = np.array([
    [0, 1],
    [-1, 0]
], dtype=np.float32)


rx = cv2.filter2D(
    img, cv2.CV_32F, roberts_x
)


ry = cv2.filter2D(
    img, cv2.CV_32F, roberts_y
)
```

```
roberts = cv2.magnitude(rx, ry)

# Prewitt operators
prewitt_x = np.array([
    [-1, 0, 1],
    [-1, 0, 1],
    [-1, 0, 1]
], dtype=np.float32)

prewitt_y = np.array([
    [-1, -1, -1],
    [ 0,  0,  0],
    [ 1,  1,  1]
], dtype=np.float32)

px = cv2.filter2D(
    img, cv2.CV_32F, prewitt_x
)

py = cv2.filter2D(
    img, cv2.CV_32F, prewitt_y
)

prewitt = cv2.magnitude(px, py)

# Sobel operators
sx = cv2.Sobel(
    img, cv2.CV_32F, 1, 0, ksize=3
)
```

```
sy = cv2.Sobel(
    img, cv2.CV_32F, 0, 1, ksize=3
)

sobel = cv2.magnitude(sx, sy)

cv2.imshow("Roberts", cv2.convertScaleAbs(roberts))
cv2.imshow("Prewitt", cv2.convertScaleAbs(prewitt))
cv2.imshow("Sobel", cv2.convertScaleAbs(sobel))

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Explanation:

- Roberts detects rapid diagonal intensity changes.
- Prewitt detects horizontal and vertical boundaries.
- Sobel uses weighted gradient masks and provides some smoothing.

C. Advanced Edge Detection – Marr-Hildreth and Canny

Marr-Hildreth uses Gaussian smoothing followed by Laplacian-based zero-crossing detection.

Canny uses smoothing, gradient detection, non-maximum suppression and hysteresis thresholding.

Program:

```
import cv2
import numpy as np

img = cv2.imread("old_photo.jpg", 0)

if img is None:
    print("Image not found")
    exit()
```

```
# Marr-Hildreth
blurred = cv2.GaussianBlur(
    img, (5, 5), 1.0
)

laplacian = cv2.Laplacian(
    blurred, cv2.CV_32F
)

marr = cv2.convertScaleAbs(laplacian)

# Canny
canny = cv2.Canny(
    img, 50, 150
)

cv2.imshow("Marr-Hildreth Approximation", marr)
cv2.imshow("Canny Edge Detection", canny)

cv2.waitKey(0)
cv2.destroyAllWindows()
```

Note: The Marr-Hildreth output above displays the Laplacian response. A complete zero-crossing implementation must additionally identify sign changes in neighbouring Laplacian values.

41. Detect dust spots, scratches, crease lines, stain boundaries and torn or missing regions.

Different damage types produce different image patterns.

Damage Type	Suitable Detection Method
Dust spots	Point detection
Thin scratches	Line masks and edge detection
Crease lines	Sobel and Canny

Stain boundaries Gradient operators

Torn boundaries Canny and boundary detection

Missing regions Boundary detection with manual review

Analysis:

Point detection is useful for small isolated spots. Line detection is suitable for narrow scratches. Gradient and advanced edge detectors identify boundaries around stains and torn areas.

However, detection results may also contain normal photograph edges, such as facial outlines and clothing boundaries. Therefore, detected regions should be reviewed before repair.

42. Compare the techniques based on noise sensitivity, false detections and missed damage regions.

Method	Noise Sensitivity	False Detections	Missed Damage
Point and Line Masks	High	May detect texture as damage	May miss different orientations
Gradient Operators	Moderate to high	May detect normal object edges	May miss weak boundaries
Canny	Lower with suitable smoothing	Can still detect unrelated edges	May miss weak or interrupted damage

Conclusion: Canny provides a useful balance of edge localization and noise control when suitable thresholds and preprocessing are selected.

43. Calculate precision and recall using manually annotated ground-truth images.

Precision and Recall are used to evaluate damage detection.

Precision:

[
Precision=\frac{TP}{TP+FP}
]

Recall:

[
Recall=\frac{TP}{TP+FN}
]

Where:

- TP = correctly detected damaged pixels.
- FP = undamaged pixels incorrectly detected as damage.
- FN = damaged pixels missed by the detector.

Program:

```
import cv2
import numpy as np

predicted = cv2.imread(
    "predicted_mask.png", 0
)

ground_truth = cv2.imread(
    "ground_truth.png", 0
)

if predicted is None or ground_truth is None:
    print("Mask image not found")
    exit()

predicted = predicted > 127
ground_truth = ground_truth > 127

tp = np.sum(predicted & ground_truth)
fp = np.sum(predicted & ~ground_truth)
fn = np.sum(~predicted & ground_truth)

precision = tp / (tp + fp + 1e-8)
recall = tp / (tp + fn + 1e-8)

print("True Positive:", tp)
print("False Positive:", fp)
print("False Negative:", fn)

print("Precision:", precision)
```

```
print("Recall:", recall)
```

Explanation:

The predicted damage mask is compared with a manually annotated ground-truth mask. The number of correctly detected, falsely detected and missed pixels is calculated.

44. Justify suitable thresholds for detecting small dust spots, thin scratches and major damaged boundaries.

Thresholds control the sensitivity of damage detection.

Damage Type	Threshold Selection
Small dust spots	Lower detection threshold
Thin scratches	Moderate threshold
Major damaged boundaries	Stronger edge response threshold

Analysis:

A low threshold detects weak edges but may also identify noise. A high threshold reduces false detections but may miss faded scratches.

The thresholds should be tested on representative photographs containing different damage types.

Conclusion: A single threshold may not work for every photograph. Threshold values should be selected based on the intensity and size of the damaged regions.

45. Analyse why directional line masks may fail when scratches and creases occur at different orientations.

Directional line masks detect lines aligned with a specific direction.

For example, a horizontal mask is designed to detect horizontal structures.

Reasons for failure:

1. A horizontal mask may miss vertical scratches.
2. Diagonal creases may not match horizontal or vertical masks.
3. Curved scratches are difficult to detect using one fixed mask.
4. Broken scratches may produce incomplete responses.
5. Complex backgrounds may generate false line detections.

Conclusion: Multiple directional masks can improve detection. Combining horizontal, vertical and diagonal responses provides better orientation coverage.

46. Compare a traditional edge detector with a deep-learning-based edge detector such as HED.

HED stands for Holistically-Nested Edge Detection. It is a deep-learning-based method that detects edges at multiple image scales.

Parameter	Traditional Edge Detector	HED
Method	Gradient and derivative operators	Deep neural network
Training	Not required	Requires pretrained weights
Edge detection	Depends on thresholds	Learns edge patterns
Complex images	May produce false edges	Can capture contextual structures
Hardware	CPU-friendly	More computationally demanding
Implementation	Simple	Requires model setup

Analysis:

Traditional detectors such as Sobel and Canny are easy to implement and suitable for a CPU-based system. HED may detect complex boundaries more effectively, but its output depends on the training data and model.

Conclusion: Canny is selected as the basic edge detection method because it provides good edge localization and is practical for the available hardware. HED may be evaluated as an advanced alternative.

Task 7 – Final Comparison and Conclusion

Technique	Main Advantage	Main Limitation
Point and Line Masks	Detect isolated spots and directional lines	Direction-dependent
Gradient Operators	Detect intensity boundaries	Sensitive to noise
Advanced Edge Detection	Better edge localization	Requires threshold selection

Selected Technique: Canny Edge Detection

Canny is selected because it combines smoothing, gradient calculation, non-maximum suppression and hysteresis thresholding. It can identify important boundaries in old photographs while reducing some unwanted noise responses.

The detected damage regions must be reviewed before restoration to avoid treating genuine facial or historical details as damage.